

Graph Laplacian for Image Deblurring

A Graph-Based Regularization Approach for Image Restoration

Raychie Zhang

July 4, 2026



Abstract

Image blurring appears in many imaging applications. Graph-based methods provide a way to incorporate image structure into restoration problems. This project introduces a graph-Laplacian regularization method for image deblurring. The report includes background on graph Laplacians, the formulation of the deblurring problem, and a numerical experiment.

Contents

1	Introduction	3
2	Background Knowledge: Graph Theory	3
3	Graph Signal Processing	5
3.1	Image as Graph	5
3.2	Weights	5
4	Deblurring Problem and Criterion	6
4.1	Model	6
4.2	Regularization	7
4.3	Minimization Problem	8
5	Optimization using Conjugate Gradient Solver	9
6	Numerical Application	10
7	Conclusion	14

1 Introduction

Image blurring is relevant in many situations. Using graphs to analysis images is a way. Graph Laplacian gives a method to deblur images that depends on the structure of the input image.

2 Background Knowledge: Graph Theory

Definition 2.1 (Graph). A **graph** G is a triple consisting of a **vertex set** $V(G)$, an **edge set** $E(G)$, and a relation that associates with each edge two vertices (not necessarily distinct) called its **endpoints**.

The elements i, j of the set V are called **vertices**.

Definition 2.2 (Simple Graph). A **loop** is an edge whose endpoints are equal. **Multiple edges** are edges having the same pair of endpoints.

A **simple graph** is a graph having no loops or multiple edges.

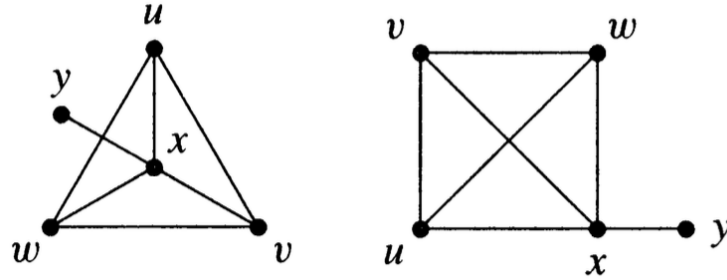


Figure 1: Two drawings of a simple graph. The vertex set is $\{u, v, w, x, y\}$, and the edge set is $\{uv, uw, ux, uy, vw, vx, wx, xy\}$.

Definition 2.3 (Complete Graph). A **complete graph** is a simple graph whose vertices are pairwise adjacent;

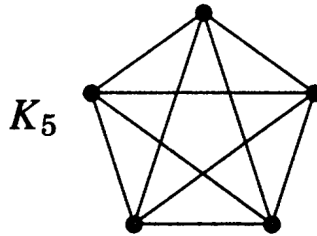


Figure 2: The (unlabeled) complete graph with 5 vertices, denoted K_5

Definition 2.4 (Adjacent). When u and v are the endpoints of an edge, i.e if there exists an edge between them, then they are **adjacent** and are **neighbors**.

Definition 2.5 (Weighted Graph). A **weighted** graph is a graph with numerical labels on the edges.

The positive value $w(i, j)$ is called weight associated to the edge $\{i, j\}$.

Definition 2.6 (Weighted Graph Laplacian). Let $e_i \in \{0, 1\}^n$ be the standard basis vectors (1 in the i -th coordinate, 0's elsewhere).

Then $(e_i - e_j)(e_i - e_j)^T$ is an $|V| \times |V|$ matrix that has +1 in the (i, i) and (j, j) coordinate, -1 in the (i, j) and (j, i) coordinate and the rest of the entries are all zero.

$$(e_i - e_j)(e_i - e_j)^T = \begin{bmatrix} 0 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & 1 & \cdots & -1 & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & -1 & \cdots & 1 & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 0 \end{bmatrix}$$

Now, we define the following notation:

- $d(i)$: **degree** of i in G .
- **adjacency matrix**: $W = (w(i, j)) \in \mathbb{R}^{n \times n}$ where $w(i, j) = 0$ if $(i, j) \notin E$
- **degree matrix**: $D = \text{diag}(d(i))$ is the $|V| \times |V|$ diagonal matrix where $D(i, i) = d(i) = \sum_{(i, j) \in E} w(i, j)$

If G has weights $w(i, j), \forall (i, j) \in E$, then the **weighted Laplacian** is

$$L_G = \sum_{(i, j) \in E} w(i, j)(e_i - e_j)(e_i - e_j)^T$$

Then $L_G = D - W$. We will sometimes denote this matrix by $L_{G, w}$.

Example 2.7. The weighted Laplacian matrix L_G of a graph G with vertices $\{1, 2, \dots, n\}$ and weights $w(i, j)$ can be defined as:

$$L_G = \begin{bmatrix} d(1) & -w(1,2) & \cdots & -w(1,n) \\ -w(2,1) & d(2) & \cdots & -w(2,n) \\ \vdots & \vdots & \ddots & \vdots \\ -w(n,1) & -w(n,2) & \cdots & d(n) \end{bmatrix}$$

where $d(i) = \sum_{k \neq i} w(i, k)$ for each i , and $w(i, j) = 0$ if there is no edge between i and j .

An interesting and useful fact is that the Laplacian L_G is positive semidefinite.

3 Graph Signal Processing

3.1 Image as Graph

We think of an image as a graph. Each pixel is a vertex and has edges connecting it to other vertices.

In principle, one may use a complete weighted graph, where every pair of pixels is connected. However, for computational efficiency, image graphs are often chosen to be sparse. In this project, the numerical implementation uses a local 4-neighbor graph.

Suppose the image has N pixels. Then the vertex set is

$$V = \{1, \dots, N\}.$$

The edge set E depends on the graph construction. For a complete graph, E contains all pairs of pixels. In the numerical implementation, E contains only 4-neighbor pixel pairs.

To preserve the image information in the graph, the graph edges will be assigned a weight, measuring the similarity between two vertices, thus between two pixels. [4]

Another Idea As suggested by Dr. Micheli, in images with prominent edges, we can instead consider patches as vertices to preserve more information on the edge structures.

3.2 Weights

Let $x : V \rightarrow \mathbb{R}$, $i \mapsto x_i$ be the function representing the intensity value of pixel i .

Let $l : V \rightarrow \mathbb{R}^2$, $i \mapsto l_i = \begin{bmatrix} r_i \\ c_i \end{bmatrix}$, where r_i is the row coordinate of i and c_i is the column coordinate of i , be the function representing the location of pixel i .

Weights A large positive edge weight $w_{i,j}$ would mean that samples at nodes i and j are expected to be similar/correlated. [2]

There are different ways to construct weights on edges. One way is by using spatial proximity. Another way is to consider the pixel’s color or intensity. A good compromise is to consider both. [4]

A common weight design for images would be to compute edge weight $w_{i,j}$ using a Gaussian kernel, as done in the bilateral filter (Tomasi and Manduchi 1998):

$$w_{i,j} = \exp\left(-\frac{\|l_i - l_j\|^2}{2\sigma_l^2}\right) \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_x^2}\right)$$

where $l_i \in \mathbb{R}^2$ is the location of pixel i on the 2D image grid, $x_i \in \mathbb{R}$ is the intensity of pixel i , and σ_l^2 and σ_x^2 are two parameters. [2]

Here, $-\frac{\|l_i - l_j\|^2}{2\sigma_l^2}$ measures the locational difference between i and j , and $-\frac{\|x_i - x_j\|^2}{2\sigma_x^2}$ measures the intensity difference.

We take exponentials for both differences to retain the value to be in $(0, 1]$. Then the resulting weight $w_{i,j} \in (0, 1]$.

We take the product of locational difference and intensity difference to measure the correlation strength between 2 vertices/pixels.

4 Deblurring Problem and Criterion

In the deblurring problem, we want the input and the output related as follows:

1. They are the “same” image.
2. The output is less blurry than the input. [3]

Notice that we need to sample the images so that the matrices won’t be too huge.

4.1 Model

Let $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \in \mathbb{R}^N$ denote a candidate sharp image to be reconstructed, where x_i is the intensity value of pixel i . The final restored image will be denoted by $\hat{\mathbf{x}}$.

Let $\mathbf{b}^\delta \in \mathbb{R}^M$ denote the observed blurred/noisy image, and let $A \in \mathbb{R}^{M \times N}$ be the linear blur operator. For a candidate restored image $\mathbf{x}$, the image predicted by the forward model is

$$A\mathbf{x}.$$

The discrepancy between the predicted blurred image and the observed data is

$$\|A\mathbf{x} - \mathbf{b}^\delta\|_2^2,$$

which is called the data fidelity term.

4.2 Regularization

We want to regularize the problem by minimizing the difference between highly similar pixels.

Consider the local term

$$w(i, j)(x_i - x_j)^2.$$

Here $w(i, j)$ denotes the similarity, or affinity, between pixels i and j , while $(x_i - x_j)^2$ measures the squared difference between their restored intensity values. The product therefore assigns a larger penalty to intensity differences between pixels that are strongly connected in the image graph. This is based on the image prior that nearby pixels within the same object or homogeneous region are likely to have similar intensities. Conversely, when two pixels are weakly related, for example across an edge or object boundary, $w(i, j)$ is small, so the regularizer does not strongly penalize a large intensity difference. Thus, the graph Laplacian term promotes smoothness within coherent regions while allowing discontinuities across edges.

Notice that for 2 pixels/vertices i and j ,

$$\begin{aligned} (e_i - e_j)^T \mathbf{x} &= e_i^T \mathbf{x} - e_j^T \mathbf{x} = x_i - x_j \\ \Leftrightarrow (x_i - x_j)^2 &= ((e_i - e_j)^T \mathbf{x})^2 \\ &= ((e_i - e_j)^T \mathbf{x})((e_i - e_j)^T \mathbf{x}) \end{aligned}$$

Since $e_i - e_j$ and $\mathbf{x}$ are both column vectors of the same dimension, we have

$$(e_i - e_j)^T \mathbf{x} = \mathbf{x}^T (e_i - e_j).$$

Thus

$$((e_i - e_j)^T \mathbf{x})((e_i - e_j)^T \mathbf{x}) = \mathbf{x}^T (e_i - e_j)(e_i - e_j)^T \mathbf{x}.$$

Since $\mathbf{x}^T \in \mathbb{R}^{1 \times N}$, $(e_i - e_j) \in \mathbb{R}^{N \times 1}$, $(e_i - e_j)^T \in \mathbb{R}^{1 \times N}$, and $\mathbf{x} \in \mathbb{R}^{N \times 1}$, we have

$$(x_i - x_j)^2 = \mathbf{x}^T (e_i - e_j)(e_i - e_j)^T \mathbf{x} \in \mathbb{R}$$

being a scalar.

Then we sum up the penalty for every pair of vertices:

$$\begin{aligned}
& \sum_{(i,j) \in E} w(i,j)(x_i - x_j)^2 \\
&= \sum_{(i,j) \in E} w(i,j) \mathbf{x}^T (e_i - e_j)(e_i - e_j)^T \mathbf{x} \\
&= \mathbf{x}^T \left(\sum_{(i,j) \in E} w(i,j)(e_i - e_j)(e_i - e_j)^T \right) \mathbf{x} \\
&= \mathbf{x}^T L_w \mathbf{x}
\end{aligned}$$

Here L_w denotes the weighted graph Laplacian associated with the image graph.

4.3 Minimization Problem

We define the restored image $\hat{\mathbf{x}}$ as the minimizer of the regularized objective:

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^N} \left\| A\mathbf{x} - \mathbf{b}^\delta \right\|_2^2 + \mu \mathbf{x}^T L_w \mathbf{x}.$$

Here $\mathbf{x}$ is the optimization variable, while $\hat{\mathbf{x}}$ denotes the final reconstructed image. The matrix A is the blur operator, $\mathbf{b}^\delta$ is the observed blurred/noisy image, L_w is the weighted graph Laplacian, and $\mu > 0$ is the regularization parameter.

The first term measures consistency with the observed data, while the second term is the graph Laplacian regularization term, which penalizes non-smoothness of the restored image on the weighted image graph.

In principle, an image can be represented by a fully connected weighted graph, where every pair of pixels is connected. However, this is computationally expensive for images with many pixels. In the numerical experiment, we use a sparse local graph with 4-neighbor pixel connections. Spatial locality is encoded by the edge set, while the edge weights encode intensity similarity.

The weights are constructed from an observed image $\mathbf{y}$, for example $\mathbf{y} = \mathbf{b}^\delta$, by

$$w_{ij} = \exp \left(-\frac{|y_i - y_j|^2}{\sigma} \right),$$

where y_i denotes the intensity of pixel i , and $\sigma > 0$ is a user-defined parameter. [1]

Define

$$f(\mathbf{x}) = \left\| A\mathbf{x} - \mathbf{b}^\delta \right\|_2^2 + \mu \mathbf{x}^T L_w \mathbf{x}.$$

Since

$$\left\| A\mathbf{x} - \mathbf{b}^\delta \right\|_2^2 = (A\mathbf{x} - \mathbf{b}^\delta)^T (A\mathbf{x} - \mathbf{b}^\delta),$$

we have

$$f(\mathbf{x}) = (A\mathbf{x} - \mathbf{b}^\delta)^\top (A\mathbf{x} - \mathbf{b}^\delta) + \mu\mathbf{x}^\top L_w \mathbf{x}.$$

Expanding the first term gives

$$f(\mathbf{x}) = \mathbf{x}^\top A^\top A \mathbf{x} - 2(\mathbf{b}^\delta)^\top A \mathbf{x} + (\mathbf{b}^\delta)^\top \mathbf{b}^\delta + \mu\mathbf{x}^\top L_w \mathbf{x}.$$

5 Optimization using Conjugate Gradient Solver

Given that we want to minimize the function

$$f(\mathbf{x}) = \mathbf{x}^\top A^\top A \mathbf{x} - 2(\mathbf{b}^\delta)^\top A \mathbf{x} + (\mathbf{b}^\delta)^\top \mathbf{b}^\delta + \mu\mathbf{x}^\top L_w \mathbf{x}.$$

Taking the gradient with respect to $\mathbf{x}$, and using $L_w = L_w^\top$, we obtain

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = 2A^\top (A\mathbf{x} - \mathbf{b}^\delta) + 2\mu L_w \mathbf{x}.$$

Here we use the standard identity

$$\nabla_{\mathbf{x}} (\mathbf{x}^\top Q \mathbf{x}) = (Q + Q^\top) \mathbf{x}.$$

Since $A^\top A$ is symmetric, we have

$$\nabla_{\mathbf{x}} (\mathbf{x}^\top A^\top A \mathbf{x}) = 2A^\top A \mathbf{x}.$$

Also, since the graph Laplacian L_w is symmetric,

$$\nabla_{\mathbf{x}} (\mathbf{x}^\top L_w \mathbf{x}) = 2L_w \mathbf{x}.$$

Finally,

$$\nabla_{\mathbf{x}} (-2(\mathbf{b}^\delta)^\top A \mathbf{x}) = -2A^\top \mathbf{b}^\delta.$$

Therefore,

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = 2A^\top A \mathbf{x} - 2A^\top \mathbf{b}^\delta + 2\mu L_w \mathbf{x} = 2A^\top (A\mathbf{x} - \mathbf{b}^\delta) + 2\mu L_w \mathbf{x}.$$

The first-order optimality condition

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = 0$$

gives

$$2A^\top (A\mathbf{x} - \mathbf{b}^\delta) + 2\mu L_w \mathbf{x} = 0.$$

Hence,

$$(A^\top A + \mu L_w) \mathbf{x} = A^\top \mathbf{b}^\delta.$$

Therefore the restored image $\hat{\mathbf{x}}$ satisfies

$$(A^\top A + \mu L_w)\hat{\mathbf{x}} = A^\top \mathbf{b}^\delta.$$

Let

$$A' = A^\top A + \mu L_w, \quad \mathbf{b}' = A^\top \mathbf{b}^\delta.$$

Then the minimization problem is reduced to solving the linear system

$$A'\hat{\mathbf{x}} = \mathbf{b}'.$$

Since A' is high-dimensional, explicitly computing the inverse $(A')^{-1}$ is computationally inefficient and numerically unnecessary. Instead, we solve this linear system iteratively using the conjugate gradient method.

6 Numerical Application

In the numerical experiment, we start from a known sharp image and use it as ground truth. Let

$$\mathbf{x}_{\text{true}} \in \mathbb{R}^N$$

denote the vectorized original downsampled image.

We generate a synthetic blurred/noisy observation by

$$\mathbf{b} = A\mathbf{x}_{\text{true}}, \quad \mathbf{b}^\delta = \mathbf{b} + \eta = A\mathbf{x}_{\text{true}} + \eta,$$

where η is additive Gaussian noise. Then $\mathbf{b}^\delta$ is treated as the observed input image, and $\mathbf{x}_{\text{true}}$ is used only for comparison with the restored image.

The restored image is computed by solving

$$(A^\top A + \mu L_w)\hat{\mathbf{x}} = A^\top \mathbf{b}^\delta.$$

Algorithm

- **Input:** Original image
- Down sample original image
- Define blurring kernel A and blurred and noised image $\mathbf{b}^\delta$
- Construct weighted graph Laplacian
- Construct optimization system
- Solve the system

Output

The restored image recovers sharper local structures than the blurred/noisy observation, while the graph Laplacian regularization helps control noise amplification.

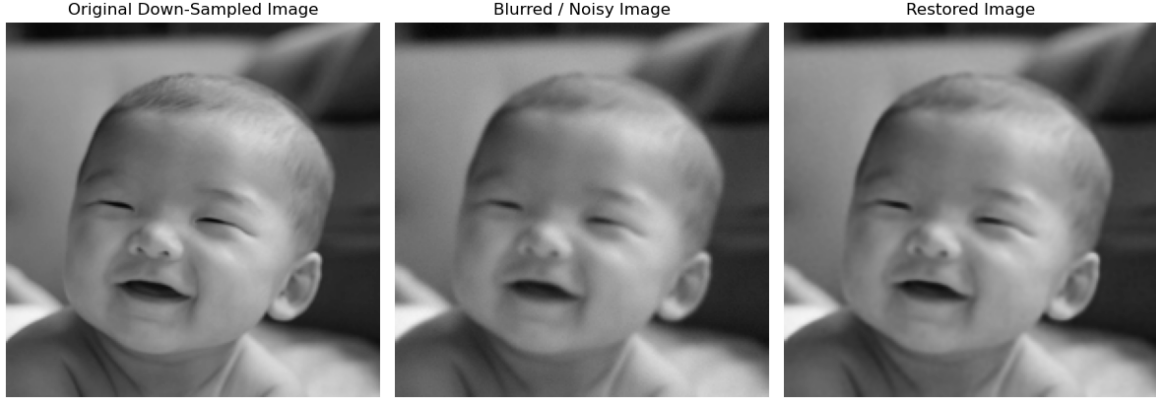


Figure 3: Numerical result of graph-Laplacian image deblurring.

Implementation outline

1. Load and downsample the grayscale image.
2. Generate the Gaussian blur kernel and blurred/noisy observation.
3. Construct the 4-neighbor weighted graph Laplacian L_w .
4. Define the linear operator

$$\mathbf{x} \mapsto A^\top A \mathbf{x} + \mu L_w \mathbf{x}.$$

5. Solve

$$(A^\top A + \mu L_w) \hat{\mathbf{x}} = A^\top \mathbf{b}^\delta$$

using conjugate gradient.

6. Display the original, blurred/noisy, and restored images.

The following code shows the core implementation of the graph Laplacian construction and the conjugate gradient solve. The full implementation is available in the accompanying GitHub repository.

```

1 from scipy.sparse import lil_matrix
2 from scipy.sparse.linalg import LinearOperator, cg
3 from scipy.signal import convolve2d
4 import numpy as np
5
6 def weighted_laplacian(image, theta=0.01):
7     """Construct a sparse weighted graph Laplacian using 4-neighbor pixels."""
8     n, m = image.shape
9     L = lil_matrix((n * m, n * m))
10
11     for i in range(n):
12         for j in range(m):
13             index = i * m + j
14
15             if i > 0:
16                 weight = np.exp(-((image[i, j] - image[i - 1, j]) ** 2) / theta)
17                 L[index, index - m] = -weight
18                 L[index, index] += weight
19
20             if i < n - 1:
21                 weight = np.exp(-((image[i, j] - image[i + 1, j]) ** 2) / theta)
22                 L[index, index + m] = -weight
23                 L[index, index] += weight
24
25             if j > 0:
26                 weight = np.exp(-((image[i, j] - image[i, j - 1]) ** 2) / theta)
27                 L[index, index - 1] = -weight
28                 L[index, index] += weight
29
30             if j < m - 1:
31                 weight = np.exp(-((image[i, j] - image[i, j + 1]) ** 2) / theta)
32                 L[index, index + 1] = -weight
33                 L[index, index] += weight
34
35     return L.tocsr()
36
37
38 # Build the graph Laplacian from the observed blurred/noisy image
39 y_for_graph = convolve2d(
40     b_delta,
41     gaussian_kernel(3, 0.8),
42     mode="same",
43     boundary="symm",
44 )
45
46 Lw = weighted_laplacian(y_for_graph)
47
48 # Right-hand side: b' = A^T b_delta
49 b_prime = convolve2d(
50     b_delta,
51     kernel.T,
52     mode="same",
53     boundary="symm",
54 )
55
56 mu = 0.1
57 n, m = b_delta.shape
58
59 def apply_system_matrix(v):
60     """Apply A'v = A^T A v + mu L_w v."""
61     x = v.reshape((n, m))
62
63     Ax = convolve2d(
64         x,

```

```

65     kernel,
66     mode="same",
67     boundary="symm",
68 )
69
70     ATAx = convolve2d(
71         Ax,
72         kernel.T,
73         mode="same",
74         boundary="symm",
75     )
76
77     Lx = Lw @ v
78
79     return ATAx.ravel() + mu * Lx
80
81
82 A_prime = LinearOperator(
83     shape=(n * m, n * m),
84     matvec=apply_system_matrix,
85     dtype=np.float64,
86 )
87
88 x_hat, info = cg(
89     A_prime,
90     b_prime.ravel(),
91     maxiter=1000,
92 )
93
94 x_hat = x_hat.reshape(n, m)
95 x_hat = np.clip(x_hat, 0, 1)

```

Listing 1: Core implementation of the graph-Laplacian deblurring solve

The omitted parts of the implementation load the image, generate the Gaussian blurred/noisy observation, compute PSNR values, and save the output figures.

7 Conclusion

The graph Laplacian regularization term provides a way to incorporate image structure into the deblurring problem.

1. The graph Laplacian regularization encodes prior knowledge about image smoothness.
2. The regularization term is flexible, since different graph constructions and edge weights can be used.
3. In the numerical experiment, the graph Laplacian term helps recover sharper local structures while controlling noise amplification.

Overall, graph Laplacian regularization balances data fidelity with graph-based smoothness. The quality of the reconstruction depends on the blur model, the graph weights, and the choice of the regularization parameter.

References

- [1] Davide Bianchi, Alessandro Buccini, Marco Donatelli, and Emma Randazzo. Graph laplacian for image deblurring. *arXiv preprint arXiv:2102.10327*, 2021.
- [2] G. Cheung and E. Magli. *Graph Spectral Image Processing*. ISTE Ltd and John Wiley Sons, Inc, London, Uk and Hoboken, USA, 2021.
- [3] Olivier L  zoray Leo Grady. *Image processing and analysis with graphs theory and practice*. Publisher Name, City, Country, 2023.
- [4] D. WOBROCK. Image processing using graph laplacian operator. *Degree Project In Computer Science And Engineering, Second Cycle*, 2019.